



Basics of SQL Server



Ishika Tiwari



C#Corner

Basics of SQL Server

Ishika Tiwari

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. Although the author/co-author and publisher have made every effort to ensure that the information in this book was correct at press time, the author/co-author and publisher do not assume and hereby disclaim any liability to any party for any loss, damage, or disruption caused by errors or omissions, whether such errors or omissions result from negligence, accident, or any other cause. The resources in this book are provided for informational purposes only and should not be used to replace the specialized training and professional judgment of a health care or mental health care professional. Neither the author/co-author nor the publisher can be held responsible for the use of the information provided within this book. Please always consult a trained professional before making any decision regarding the treatment of yourself or others.

Author – Ishika Tiwari

Publisher – C# Corner

Editorial Team – Deepak Tewatia, Baibhav Kumar

Publishing Team – Praveen Kumar

Promotional & Media – Rohit Tomar

Table of Contents:

Introduction to SQL Server	4
SQL Server Management Studio (SSMS)	6
Basics of SQL Server.....	11
Creating and Managing Databases	14
Tables and Relationships.....	17
Data Manipulation	21
Advanced SQL Queries	23
Stored Procedures and Functions.....	33
Database backup With Table and Data	40

1

Introduction to SQL Server

Overview

In this chapter, we explore the foundational aspects of SQL Server, delving into its definition, historical evolution, and key features. Prepare yourself for a comprehensive journey that will empower you to effectively utilize SQL Server's robust capabilities in your applications.

What is SQL Server?

Microsoft created SQL Server, a relational database management system (RDBMS). It is intended to store and retrieve information that other software programs, whether they operate on the same computer or on a different machine connected to a network, request. Because of its great scalability and support for a wide range of data operations, SQL Server is appropriate for applications ranging from small to corporate.

History and Evolution

Microsoft debuted SQL Server in 1989 through a collaboration with Sybase. Over the years, it has undergone significant transformations, with each iteration introducing novel features, bolstering security, and refining efficiency. Here are some key milestones in its development:

- **SQL Server 2000:** Paved the way for smoother interaction with web-based applications by introducing support for XML and HTTP.
- **SQL Server 2005:** Marked the introduction of SQL Server Management Studio (SSMS), dynamic management views, and enhanced XML capabilities.
- **SQL Server 2008:** Ushered in advancements like policy-based management, novel data types, and improved performance.
- **SQL Server 2012:** Introduced AlwaysOn Availability Groups, streamlined integration services, and fortified security features.
- **SQL Server 2016:** Broadened its reach by offering support for Linux, introduced real-time operational analytics, and incorporated new security features like Always Encrypted.
- **SQL Server 2019:** Added functionalities like big data clusters, data virtualization, and further advancements in intelligent query processing.
- **SQL Server 2022 (latest version):** Introduced features focused on enhanced performance, security, and manageability. Some of these include Accelerated Database Recovery, Online Index Rebuild, and Confidential Ledger.

Key Features

- **High Availability:** Always On Availability Groups and other similar solutions make sure databases are highly available.
- **Security:** Sensitive data is protected by advanced security features like row-level security, always encrypted, and transparent data encryption.
- **Performance:** Database performance can be enhanced with the use of tools such as the Query Store and performance tuning advisers.
- **Scalability:** SQL Server has the capacity to process massive volumes of data and thousands of concurrent users.
- **Integration:** SQL Server Reporting Services (SSRS) offers strong reporting capabilities, while SQL Server Integration Services (SSIS) enables reliable ETL procedures.
- **Cloud Support:** SQL Server can be used in hybrid cloud settings because of its smooth integration with cloud services such as Microsoft Azure.

2

SQL Server Management Studio (SSMS)

Overview

In this chapter, we delve into SQL Server Management Studio (SSMS), a powerful integrated environment for managing SQL Server infrastructure. We will cover the essential aspects of connecting to SQL Server, provide an overview of the SSMS interface, and explore its key components. Prepare yourself for a comprehensive journey that will empower you to efficiently configure, monitor, and manage your SQL Server instances using SSMS.

Introduction to SSMS

For administering SQL infrastructure, SQL Server Management Studio (SSMS) offers a potent integrated environment. It offers a suite of tools for configuring, monitoring, and managing SQL Server and database instances in addition to an easy graphical user interface. With its tools for query execution, database architecture, and performance tuning, SSMS is a must for database developers, administrators, and analysts.

Connecting to SQL Server

Connecting to SQL Server typically involves using a client application or programming language that supports SQL Server connections. Here's a general guide on how to connect to SQL Server:

Choose a Client Application or Programming Language:

- You can connect to SQL Server using various client applications such as SQL Server Management Studio (SSMS), Azure Data Studio, or programming languages like C#, Java, Python, etc.

Collect Connection Information:

- **Server Name or IP Address:** The address of the SQL Server instance you want to connect to.
- **Authentication Method:** SQL Server supports Windows Authentication (integrated security) and SQL Server Authentication (username/password).
- **Username and Password:** Required for SQL Server Authentication.

Using SQL Server Management Studio (SSMS):

- Launch SSMS and provide the necessary connection information.
- Open SSMS.
- In the "Connect to Server" dialog, enter the server's name and authentication method.
- Click "Connect".

Using Programming Languages:

- Install the necessary SQL Server client libraries or drivers for your programming language.
- Use the appropriate connection string to establish a connection. Here example:

```
C# with SqlConnection:

using System.Data.SqlClient;
using System;

class Program
{
    static void Main()
    {
```

```
string connectionString =
"Server=myServerAddress;Database=myDataBase;User
Id=myUsername;Password=myPassword;";
using (SqlConnection connection = new
SqlConnection(connectionString))
{
    try
    {
        connection.Open();
        Console.WriteLine("Connected to SQL Server");
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
}
```

Testing the Connection:

- Once connected, you can test the connection by running queries or performing database operations.

Connecting to SQL Server is a crucial step in managing databases and performing data operations. Make sure to handle connection strings and credentials securely to prevent unauthorized access.

SSMS Interface Overview

SQL Server Management Studio (SSMS) is a comprehensive tool for managing SQL Server databases. It provides an intuitive graphical interface that makes it easier to manage databases, run queries, and perform administrative tasks. Here's a beginner-friendly overview of the SSMS interface.

Key Components of SSMS:

- Object Explorer
- Query Editor
- Properties Window
- Solution Explorer
- Registered Servers
- Template Explorer

Let's explore each of these components in detail.

Object Explorer

Location: On the left side of the SSMS window.

Purpose: Provides a hierarchical view of your SQL Server instances and databases. It is the primary navigation tool for managing server objects.

Key Functions:

- **Databases:** View and manage all databases on the connected server.
- **Security:** Manage server-level security objects such as logins, server roles, and credentials.
- **Server Objects:** Manage linked servers, triggers, and backup devices.
- **Replication:** Set up and manage database replication.
- **Management:** Access server management tools, including SQL Server Agent and Policy Management.

Example Usage:

- Expand the "Databases" node to see a list of databases.
- Right-click on a database to create a new table, view properties, or perform other tasks.

Query Editor

Location: The central area of the SSMS window, opens when you start a new query.

Purpose: Allows you to write and execute SQL queries.

Key Features: Syntax Highlighting: Differentiates SQL keywords, functions, and variables with colours for easier reading.

IntelliSense: Provides code completion suggestions and syntax error detection.

Results Pane: Displays query results below the query editor.

Example Usage:

- Open a new query window by right-clicking on a database in Object Explorer and selecting "New Query".
- Write a query (e.g., `SELECT * FROM Employees;`).
- Execute the query by clicking the "Execute" button or pressing F5.

Properties Window

Location: Typically found at the bottom right of the SSMS window.

Purpose: Displays detailed properties of the selected object in Object Explorer. Allows you to view and edit object properties.

Example Usage:

- Select a table in Object Explorer.
- View and modify properties such as column names, data types, and constraints in the Properties window.

Solution Explorer

Location: Can be opened from the "View" menu.

Purpose: Helps you manage SQL Server projects and organize scripts and other files into solutions.

Example Usage:

- Create a new project to organize your SQL scripts.
- Add existing scripts to your project for easy access and management.

Registered Servers

Location: Can be accessed from the "View" menu.

Purpose: Allows you to register and manage multiple SQL Server instances. Useful for administrators managing multiple servers.

Example Usage:

- Register a new server by right-clicking on "Local Server Groups" and selecting "New Server Registration".
- Switch between different registered servers for management tasks.

Template Explorer

Location: Can be accessed from the "View" menu or by pressing Ctrl+Alt+T.

Purpose: Provides a collection of predefined SQL templates for common tasks. These templates can be customized and used as starting points for your SQL scripts.

Example Usage:

- Open Template Explorer to view available templates.
- Double-click on a template (e.g., "Create Table") to open it in the Query Editor.
- Modify the template as needed and execute the script.

3

Basics of SQL Server

Overview

In this chapter, we explore SQL Server data types and basic SQL commands, providing a comprehensive guide to defining and manipulating data within your database. Prepare yourself for an in-depth understanding of how to categorize and manage data efficiently, as well as how to perform essential database operations using SQL commands.

Data Types

In SQL Server, data types are essential for defining the kind of data that can be stored in each column of a table. Here's an overview of all the data types available in SQL Server:

Numeric Data Types:

- **BIT**: Represents a single bit (0 or 1) of data.
- **TINYINT**: Integer data type with values ranging from 0 to 255.
- **SMALLINT**: Integer data type with values ranging from -32,768 to 32,767.
- **INT**: Integer data type with values ranging from -2,147,483,648 to 2,147,483,647.
- **BIGINT**: Integer data type with values ranging from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807.
- **DECIMAL(p, s)**: Fixed-point decimal numbers where p is the precision (total number of digits) and s is the scale (number of digits to the right of the decimal point).
- **SMALLMONEY**: Monetary data type with values ranging from -214,748.3648 to 214,748.3647.
- **MONEY**: Monetary data type with values ranging from -922,337,203,685,477.5808 to 922,337,203,685,477.5807.

Approximate Numeric Data Types:

- **FLOAT(n)**: Floating-point number with the user-specified precision n.
- **REAL**: Floating-point number with 7-digit precision.

Date and Time Data Types:

- **DATE**: Date data type representing year, month, and day.
- **TIME(p)**: Time data type representing time of day with optional precision p.
- **DATETIME**: Date and time data type representing year, month, day, hour, minute, and second.
- **SMALLDATETIME**: Date and time data type with values ranging from January 1, 1900, to June 6, 2079, with accuracy to the minute.
- **DATETIME2(p)**: Date and time data type with user-specified precision.
- **DATETIMEOFFSET(p)**: Date and time data type with time zone offset and user-specified precision.

Character String Data Types:

- **CHAR(n)**: Fixed-length character string with a maximum length of n characters.
- **VARCHAR(n)**: Variable-length character string with a maximum length of n characters.
- **TEXT**: Variable-length character string for large text data (deprecated, use VARCHAR(MAX)).
- **NCHAR(n)**: Fixed-length Unicode string with a maximum length of n characters.
- **NVARCHAR(n)**: Variable-length Unicode string with a maximum length of n characters.
- **NTEXT**: Variable-length Unicode string for large text data (deprecated, use NVARCHAR(MAX)).

Binary Data Types:

- **BINARY(n)**: Fixed-length binary data with a maximum length of n bytes.

- **VARBINARY(n):** Variable-length binary data with a maximum length of n bytes.
- **IMAGE:** Variable-length binary data for large binary objects (deprecated, use VARBINARY(MAX)).

Basic SQL Commands

Basic SQL Commands in SQL Server:

SQL Server supports standard SQL commands along with some proprietary extensions. Here are some basic SQL commands in SQL Server:

DDL (Data Definition Language):

- **CREATE DATABASE:** Creates a new database.
- **CREATE TABLE:** Creates a new table.
- **ALTER TABLE:** Modifies an existing table's structure.
- **DROP TABLE:** Deletes a table from the database.

DML (Data Manipulation Language):

- **SELECT:** Retrieves data from one or more tables.
- **INSERT INTO:** Adds new records into a table.
- **UPDATE:** Modifies existing records in a table.
- **DELETE:** Removes records from a table.

DCL (Data Control Language):

- **GRANT:** Provides specific privileges to users.
- **REVOKE:** Removes privileges granted to users.

TCL (Transaction Control Language):

- **COMMIT:** Saves all changes made in the current transaction.
- **ROLLBACK:** Reverts changes made in the current transaction.
- **SAVE TRANSACTION:** Sets a save point within a transaction to which you can later roll back.

4

Creating and Managing Databases

Overview

In this chapter, we delve into the fundamentals of creating and managing databases in SQL Server. You'll learn how to set up your first database, modify its properties, manage database files, perform backup and restore operations, and ensure database security. This chapter provides a solid foundation for beginners to start working effectively with SQL Server databases.

In this chapter, we'll delve into the fundamental concepts of creating and managing databases in SQL Server, catering specifically to beginners. Whether you're new to SQL Server or database management in general, this chapter will provide you with a solid foundation to start working with databases effectively.

Understanding Databases

Let's begin by understanding what databases are and why they are crucial in the world of information technology. Simply put, a database is a structured collection of data that is organized and stored for easy access, retrieval, and management. Think of it as a digital filing cabinet where you can store and organize your data in a systematic manner.

Getting Started with SQL Server

SQL Server is a popular relational database management system (RDBMS) developed by Microsoft. It provides a robust platform for creating, managing, and querying databases. Before we dive into creating databases, let's ensure you have SQL Server installed and ready to use. You can download and install SQL Server Express edition for free, which is suitable for learning and small-scale applications.

Creating Your First Database

Now that you have SQL Server set up, let's create your first database. We'll use SQL Server Management Studio (SSMS), a graphical tool provided by Microsoft, to interact with SQL Server. Launch SSMS, connect to your SQL Server instance, and let's create a new database.

- Open SSMS and connect to your SQL Server instance.
- Right-click on "Databases" in the Object Explorer pane.
- Select "New Database" and provide a name for your database.
- Optionally, you can specify additional settings such as the filegroup and collation.
- Click "OK" to create the database.

Congratulations! You've successfully created your first database in SQL Server.

Managing Your Database

Once you've created a database, it's essential to understand how to manage it effectively. This includes tasks such as modifying database properties, adding and removing files, and ensuring data integrity through regular backups and restores.

Modifying Database Properties

You can modify various properties of your database using SSMS or by executing SQL statements. This includes changing the database collation, adjusting filegroup settings, or altering database options.

Adding and Removing Files

Database files, such as the primary data file (MDF) and log file (LDF), play a crucial role in storing your data. You can add or remove database files to manage storage allocation and optimize performance.

Backup and Restore Operations

Backups are essential for protecting your data against loss or corruption. SQL Server provides various backup types, such as full, differential, and transaction log backups. You can create backup jobs to automate the backup process and perform restores when necessary.

Monitoring and Maintenance

Regular monitoring and maintenance are vital for ensuring the health and performance of your database. SQL Server offers built-in tools for monitoring database activity, checking database integrity, and optimizing performance through index maintenance and query tuning.

Database Security

Database security is a critical aspect of database management. You can control access to your database by managing permissions, roles, and users. Additionally, you can implement encryption to protect sensitive data and audit database activities for compliance purposes.

5

Tables and Relationships

Overview

In this chapter, we delve into the essentials of creating and managing tables in SQL Server. We will cover key concepts such as defining various types of keys, including primary, foreign, unique, composite, alternate, and surrogate keys, each crucial for establishing relationships and ensuring data integrity. Additionally, we will explore the principles of normalization, a process that structures database tables to minimize redundancy and enhance data integrity. Prepare yourself for a comprehensive journey that will empower you to effectively organize and optimize your database structures in SQL Server.

Creating Tables

In SQL Server, you use the CREATE TABLE statement to create a new table. Here's a basic example:

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY,  
    UserName NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100)  
);
```

This creates a table named Users with columns UserID, UserName, and Email. UserID is defined as the primary key.

Keys in SQL Server

In SQL Server, there are several types of keys that are used to establish relationships between tables and ensure data integrity. Here's an explanation of each key type:

Primary Key

- **Definition:** A primary key uniquely identifies each record in a table. It must contain unique values and cannot contain NULL values.
- **Syntax:** Defined when creating a table using the PRIMARY KEY constraint.

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY,  
    UserName NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100)  
);
```

Foreign Key

- **Definition:** A foreign key establishes a link between two tables. It ensures referential integrity by enforcing a link between a column in one table and a primary or unique key in another table.
- **Syntax:** Defined using the FOREIGN KEY constraint.

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    OrderDate DATE,  
    UserID INT,  
    FOREIGN KEY (UserID) REFERENCES Users(UserID)  
);
```

In this example, UserID in the Orders table references the UserID column in the Users table.

Unique Key

- **Definition:** A unique key ensures that all values in a column or a set of columns are unique (no duplicates), like a primary key but does allow NULL values (in SQL Server, a unique constraint allows one NULL value).
- **Syntax:** Defined using the UNIQUE constraint.

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    ProductName NVARCHAR(50) NOT NULL,  
    SKU NVARCHAR(20) UNIQUE  
);
```

In this example, SKU is a unique key for the Products table.

Composite Key

- **Definition:** A composite key consists of multiple columns that together uniquely identify a record in a table.
- **Syntax:** Defined by including multiple columns in the PRIMARY KEY constraint.

```
CREATE TABLE OrderDetails (  
    OrderID INT,  
    ProductID INT,  
    Quantity INT,  
    PRIMARY KEY (OrderID, ProductID),  
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

In this example, (OrderID, ProductID) form a composite primary key in the OrderDetails table.

Alternate Key

- **Definition:** An alternate key is a column or set of columns that can also uniquely identify a record in a table, similar to a candidate key.
- **Example:** If a table has both a primary key and another unique key, the unique key is an alternate key.

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Email NVARCHAR(100) UNIQUE,  
    Phone NVARCHAR(20) UNIQUE  
);
```

In this example, both Email and Phone can serve as alternate keys for the Customers table.

Surrogate Key

- **Definition:** A surrogate key is a unique identifier generated by the database system, often an auto-incremented integer, to uniquely identify each row in a table.

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY IDENTITY,  
    FirstName NVARCHAR(50),  
    LastName NVARCHAR(50),  
    ...  
);
```

Here, EmployeeID is a surrogate key generated automatically by SQL Server.

These key types play crucial roles in defining relationships, ensuring data integrity, and optimizing database performance in SQL Server.

Normalization

Normalization in SQL Server involves structuring database tables to minimize redundancy and dependency, enhancing data integrity and query performance. This process typically includes organizing data into separate tables and establishing relationships using primary keys, foreign keys, and normalization forms such as 1NF, 2NF, and 3NF. By eliminating duplicate data and ensuring each table stores information relevant to a single entity, normalization optimizes storage efficiency and reduces the risk of anomalies during data modification and retrieval operations.

6

Data Manipulation

Overview

In this chapter, we delve into data manipulation in SQL Server, covering the essential operations of inserting, updating, deleting, and selecting data. Prepare yourself for a comprehensive journey that will empower you to effectively manage and manipulate your database records using SQL Server syntax, ensuring you can add, modify, remove, and retrieve data efficiently.

Data Manipulation covers essential operations such as inserting, updating, deleting, and selecting data. Here's a brief explanation of each using SQL Server syntax:

Inserting Data

To insert data into a table, use the INSERT INTO statement:

```
INSERT INTO TableName (Column1, Column2, ...)
VALUES (Value1, Value2, ...);
```

Example:

```
INSERT INTO Users (UserName, Email)
VALUES ('JohnDoe', 'johndoe@example.com');
```

Updating Data

To update existing data in a table, use the UPDATE statement:

```
UPDATE TableName
SET Column1 = Value1, Column2 = Value2, ...
WHERE Condition;
```

Example:

```
UPDATE Users
SET Email = 'newemail@example.com'
WHERE UserID = 1;
```

Deleting Data

To delete data from a table, use the DELETE FROM statement:

```
DELETE FROM TableName
WHERE Condition;
```

Example:

```
DELETE FROM Users
WHERE UserID = 1;
```

Selecting Data

To retrieve data from a table, use the SELECT statement:

```
SELECT Column1, Column2, ...
FROM TableName
WHERE Condition;
```

Example:

```
SELECT * FROM Users;
```

These operations are fundamental for manipulating data in SQL Server, allowing you to add, modify, remove, and retrieve records efficiently based on specified conditions.

7

Advanced SQL Queries

Overview

In this chapter, we delve into the intricacies of SQL Server joins, aggregate functions, subqueries, and indexes. Prepare yourself for a detailed exploration of these fundamental concepts, which will empower you to efficiently retrieve, manipulate, and manage data within your SQL Server databases. You'll learn how to use various types of joins to combine data from multiple tables, apply aggregate functions to summarize data, implement subqueries for complex data retrieval, and create indexes to optimize query performance.

Joins

Joins in SQL Server are used to retrieve data from two or more tables based on a related column between them. Here are the main types of joins, along with examples for each:

INNER JOIN

The INNER JOIN keyword selects records that have matching values in both tables.

Example: Suppose we have two tables, Customers and Orders:

Customers Table:

CustomerID	CustomerName
1	John Doe
2	Jane Smith
3	Sam Johnson

Orders Table:

OrderID	CustomerID	OrderDate
101	1	2023-01-01
102	2	2023-01-02
103	1	2023-01-03

Query:

```
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID,
Orders.OrderDate
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerID	CustomerName	OrderID	OrderDate
1	John Doe	101	2023-01-01
1	John Doe	103	2023-01-03
2	Jane Smith	102	2023-01-02

LEFT JOIN (or LEFT OUTER JOIN)

The LEFT JOIN keyword returns all records from the left table (Customers), and the matched records from the right table (Orders). The result is NULL from the right side, if there is no match.

Query:

```
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID,
Orders.OrderDate
FROM Customers
```



```
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerID	CustomerName	OrderID	OrderDate
1	John Doe	101	2023-01-01
1	John Doe	103	2023-01-03
2	Jane Smith	102	2023-01-02
3	Sam Johnson	NULL	NULL

RIGHT JOIN (or RIGHT OUTER JOIN)

The RIGHT JOIN keyword returns all records from the right table (Orders), and the matched records from the left table (Customers). The result is NULL from the left side, when there is no match.

Query:

```
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID,  
Orders.OrderDate  
FROM Customers  
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerID	CustomerName	OrderID	OrderDate
1	John Doe	101	2023-01-01
2	Jane Smith	102	2023-01-02
1	John Doe	103	2023-01-03

FULL JOIN (or FULL OUTER JOIN)

The FULL JOIN keyword returns all records when there is a match in either left (Customers) or right (Orders) table records. If there is no match, the result is NULL from the side where there is no match.

Query:

```
SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID,  
Orders.OrderDate  
FROM Customers  
FULL JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result:

CustomerID	CustomerName	OrderID	OrderDate
1	John Doe	101	2023-01-01
1	John Doe	103	2023-01-03
2	Jane Smith	102	2023-01-02

CustomerID	CustomerName	OrderID	OrderDate
3	Sam Johnson	NULL	NULL
NULL	NULL	104	2023-01-04

CROSS JOIN

The CROSS-JOIN keyword returns the Cartesian product of the two tables. This means that it returns all possible combinations of rows from both tables.

Query:

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
CROSS JOIN Orders;
```

Result:

CustomerName	OrderID
John Doe	101
John Doe	102
John Doe	103
Jane Smith	101
Jane Smith	102
Jane Smith	103
Sam Johnson	101
Sam Johnson	102
Sam Johnson	103

SELF JOIN

A SELF JOIN is a regular join, but the table is joined with itself.

Example: Suppose we have a table Employees with a ManagerID that references the EmployeeID of their manager.

Employees Table:

EmployeeID	EmployeeName	ManagerID
1	John Doe	NULL
2	Jane Smith	1
3	Sam Johnson	1
4	Anna Brown	2

Query:

```
SELECT e1.EmployeeName AS Employee, e2.EmployeeName AS Manager
```

```
FROM Employees e1  
LEFT JOIN Employees e2 ON e1.ManagerID = e2.EmployeeID;
```

Result:

Employee	Manager
John Doe	NULL
Jane Smith	John Doe
Sam Johnson	John Doe
Anna Brown	Jane Smith

These examples cover the main types of joins in SQL Server. Each type of join serves a different purpose and can be used depending on the requirements of your query.

Aggregate Functions

Aggregate functions in SQL are functions that perform calculations on a set of values and return a single value as a result. These functions are commonly used in conjunction with the SELECT statement to summarize data from tables. Here's an overview of some common aggregate functions

Except for COUNT(*), all aggregation functions in SQL Server are built-in functions that do not accept NULL data. An aggregate function returns a single value after calculating a set of values. These functions are mostly used in database query languages' SELECT statements' GROUP BY and HAVING clauses.

It must be noted that the aggregate functions cannot be nested, and that the expression cannot be a subquery.

When do we use Aggregate functions?

- The SELECT statement's select list (either a subquery or an outer query).
- A HAVING clause.

The following aggregate functions are available in SQL Server.

- SUM
- MAX
- MIN
- AVG
- COUNT
- APPROX_COUNT_DISTINCT
- CHECKSUM_AGG
- GROUPING
- GROUPING_ID
- VAR
- VARP
- COUNT_BIG
- STDEV
- STDEVP
- STRING_AGG

Note: To understand these aggregate functions, create a table in SQL Server name as simple.

	S.no	FuritsName	Amount
1	1	Apple	150
2	2	Banana	75
3	3	Orange	60
4	4	Papaya	80
5	5	Mango	95
6	6	Apple	150
7	7	lechi	NULL

SUM function in SQL Server

This function disregards NULL values. It is an integrated function that takes a single parameter, a column, or a valid expression and outputs a single result that condenses the incoming data set. When the result set doesn't include any rows, it returns NULL. The WHERE, GROUP BY, ORDER BY, and HAVING clauses can also acquire the filtered output.

```
SELECT SUM(Amount) AS TotalAmount FROM example
```

Result

	TotalAmount
1	610

MAX function in SQL Server

Aggregate functions include such like SQL Server's MAX() function. It is used to find the highest or largest value among a set of values for a certain column or expression. The WHERE, GROUP BY, ORDER BY, and HAVING clauses can also acquire the filtered output.

```
SELECT MAX(Amount) AS MaxPrice FROM example
```

Result

	MaxPrice
1	150

MIN function in SQL Server

Aggregate functions include the MIN() function in SQL Server. It determines the lowest or least value for a given column or expression. The WHERE, GROUP BY, ORDER BY, and HAVING clauses can also acquire the filtered output.

```
SELECT MIN(Amount) AS MinPrice FROM example
```

Result

	MinPrice
1	60

AVG function in SQL Server

A built-in function called AVG() is frequently used with the SELECT statement. The average value of an expression is determined with this function. This function ignores the values for NULL. The WHERE, GROUP BY, ORDER BY, and HAVING clauses can also acquire the filtered output.

```
SELECT AVG(Amount) AS average_price FROM example
```

Result

	average_price
1	101

COUNT function in SQL Server

A SQL Server function called COUNT counts the number of rows in the table or result set supplied or returned by a SELECT statement. The COUNT function returns the quantity of non-null values included in the specified column or expression when given the column's name or expression as an argument. COUNT returns the overall number of rows in the table when used with the wildcard character asterisk (*).

```
SELECT COUNT(Amount) AS count_item FROM example
```

Result

	count_item
1	6

APPROX_COUNT_DISTINCT function in SQL Server

A SQL Server function called APPROX_COUNT_DISTINCT calculates an approximation of the number of distinct values in a given column or expression. Based on a statistical sample of the data, APPROX_COUNT_DISTINCT employs a probabilistic approach to calculate the approximate number of unique values. When the precise number of unique values is not required, it is a quicker option to COUNT unique.

```
SELECT APPROX_COUNT_DISTINCT(amount) AS APPROX_DISTINCT FROM example
```

Result

	APPROX_DISTINCT
1	5

VAR function in SQL Server

A SQL Server function called VAR can determine the variance of a set of values in a given column or expression. It is a statistical function that assesses how variable or dispersed a dataset is. Variance is calculated as the sum of the squared deviations between each value and the mean divided by the total number of values less than 1. The VAR function receives the name of a column or expression as an argument and outputs the variance of the values in that column or expression.

```
SELECT VAR(amount) AS var_value FROM example
```

Result

	var_value
1	1526.66666666667

VARP function in SQL Server

A statistical function called VARP in SQL Server provides the variance of a collection of integers. The VARP function returns the values' variance in the column or expression that has been supplied. The variance measures how far apart the values are from the average value. The variance is aThe values are more evenly distributed when the variation is bigger, and they are more closely spaced when the variance is lower.

```
SELECT VARP(amount) AS varp_value FROM example
```

Result

	varp_value
1	1272.22222222222

COUNT_BIG function in SQL Server

The built-in COUNT_BIG function in SQL Server counts the number of rows in a given table or view. It is similar to the COUNT function but is used specifically for larger data sets when the output can be greater than the $2^{31} - 1$ maximum value of the standard COUNT function. Use COUNT_BIG rather than COUNT if you are working with massive data sets and need to count the number of rows to avoid integer overflow problems.

```
SELECT COUNT_BIG(amount) AS bigcount_value FROM example
```

Result

	bigcount_value
1	6

STDEV function in SQL Server

The built-in statistical function STDEV in SQL Server determines the standard deviation of a collection of numerical values in a given table column. It determines the degree of variance or dispersion among the column's data points.

```
SELECT STDEV(amount) AS stdev_value FROM example
```

Result

	stdev_value
1	39.0725820322469

STDEVP function in SQL Server

The built-in function STDEVP in SQL Server determines a population's standard deviation. It accepts a collection of numerical inputs and outputs the population's overall standard deviation. Remember that STDEVP delivers the population-wide standard deviation, not simply a sample. Instead, the STDEV function would be best to determine a sample's standard deviation.

```
SELECT STDEVP(amount) AS stdevp_value FROM example
```

Result

	stdevp_value
1	35.6682242650545

STRING_AGG function in SQL Server

The STRING_AGG String Function in SQL Server joins the string expressions and inserts a designated separator between them, remembering that it won't add a separator to the string's end. The WHERE, GROUP BY, ORDER BY, and HAVING clauses can also acquire the filtered output.

```
SELECT STRING_AGG(amount, '-') AS col_row FROM example
```

Result

	col_row
1	150-75-60-80-95-150

Subqueries

A subquery, also known as a nested query or inner query, is a query nested within another SQL statement. It allows you to execute a query inside another query. Subqueries can be used in various SQL statements such as SELECT, INSERT, UPDATE, or DELETE to retrieve or manipulate data.

```
SELECT column1, column2
FROM table1
WHERE column1 IN (SELECT column1 FROM table2 WHERE condition);
```

In this example, (SELECT column1 FROM table2 WHERE condition) is a subquery that retrieves data from table2 based on a condition, and the outer query uses this result to filter rows from table1.

Indexes

Indexes in SQL Server are database objects that improve the speed of data retrieval operations on database tables at the cost of additional storage space and decrease performance on data modification operations (such as INSERT, UPDATE, DELETE). Indexes are created on columns in tables or views.

Types of indexes in SQL Server:

- **Clustered Index:** Determines the physical order of data rows in a table. Each table can have only one clustered index because the data rows themselves can only be sorted in one order.

```
CREATE CLUSTERED INDEX idx_EmployeeID ON Employees(EmployeeID);
```

In this example, EmployeeID is likely a unique identifier for each employee. By creating a clustered index on EmployeeID, SQL Server physically orders the rows in the Employees table by EmployeeID.

- **Non-clustered Index:** A separate structure from the data rows that contains pointers to the data rows in a specific order, based on the indexed column(s).

```
CREATE NONCLUSTERED INDEX idx_LastName ON Employees (LastName);
```

This creates a non-clustered index on the LastName column. Non-clustered indexes are useful for columns that are frequently used in search conditions or join operations.

- **Unique Index:** Ensures that the indexed columns contain unique values, similar to a clustered or non-clustered index but with the uniqueness constraint.

```
CREATE UNIQUE INDEX idx_Email ON Employees(Email);
```

This ensures that each email address in the Employees table is unique. Attempting to insert or update a row with a duplicate email address would result in an error.

Indexes are crucial for improving query performance, especially for tables with large amounts of data.

8

Stored Procedures and Functions

Overview

In this chapter, we explore SQL Server stored procedures and custom functions, fundamental components for efficient database management and data manipulation. You'll learn how to create, use, and manage stored procedures for performing CRUD (Create, Read, Update, Delete) operations on your database tables. Additionally, we'll delve into creating custom scalar-valued and table-valued functions to extend the capabilities of SQL Server. Prepare yourself for a detailed journey that will empower you to leverage the power of stored procedures and custom functions effectively in SQL Server.

Introduction to Stored Procedures

SQL Server stored procedures are robust database objects that hold one or more SQL statements. These processes are effective and reusable because they are assembled and saved within the database itself. An overview of SQL Server stored procedures is given here:

Definition

A set of SQL statements used to carry out a particular operation is called a stored procedure. It can execute other stored procedures, receive input arguments, execute operations like SELECT, INSERT, UPDATE, and DELETE, and return output parameters or result sets.

Advantages:

- **Code Reusability:** Once created, stored procedures can be executed multiple times without rewriting the same code.
- **Performance:** Stored procedures are precompiled and stored in the database, reducing overheads and improving performance.
- **Security:** You can grant permissions to execute stored procedures without giving direct table access to users.
- **Maintainability:** Changes can be made to a single stored procedure rather than updating the same logic in multiple places.

Creating Stored Procedures

How can we create an insert stored procedure in the SQL server?

To add new records to the database, utilize the Create operation. To develop a stored procedure

```
CREATE PROCEDURE InsertProject
    @ProjectName NVARCHAR(500),
    @ClientName NVARCHAR(500),
    @ProjectManagerId NVARCHAR(500),
    @ProjectDescription NVARCHAR(500) = NULL,
    @StartDate DATETIME
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Projects
    (
        [ProjectName],
        [ClientName],
        [ProjectManagerId],
        [CreatedDate],
        [ProjectDescription],
        [StartDate]
    )
    VALUES
    (
        @ProjectName,
```

```

        @ClientName,
        @ProjectManagerId,
        GETUTCDATE(),
        @ProjectDescription,
        @StartDate
    )
END
EXEC InsertProject
    @ProjectName = 'Management Software',
    @ClientName = 'Shubham Mishra',
    @ProjectManagerId = 'A51DC085-073F-4D3A-AFC8-ACE61B89E8C8',
    @ProjectDescription = 'This is a sample Management Software.',
    @StartDate = '2023-01-20 12:45:00.000';

```

How can we create a Read stored procedure in the SQL server?

Retrieving data from the database is done using the Read operation. To develop a select stored procedure, you can use a simple SELECT statement without any constraints to get all the data from a table within a stored procedure.

```

CREATE PROCEDURE GetAllProject
AS
BEGIN
    SET NOCOUNT ON;
    SELECT Id, ProjectName,
    ClientName, ProjectDescription, StartDate, EndDate, CreatedDate from
    Projects
END
EXEC GetAllProject

```

Output

	Id	ProjectName	ClientName	ProjectManagerId	ProjectDescription	StartDate	EndDate	CreatedDa...
1	1	Management ...	Shubham Mishra	A51DC085-073F-...	This is a sample ...	2023-01-...	NULL	2023-07-...
2	2	Go pure	Dinesh sir	A51DC085-073F-...	Go pure Descripti...	2023-01-...	NULL	2023-07-...
3	3	Timesheet	Bhasker sir	A51DC085-073F-...	This a sample Ti...	2023-01-...	NULL	2023-07-...

Use a WHERE clause with the proper condition to retrieve a specific row from the table using a unique identifier (such as Id) within a stored procedure.

```

CREATE PROCEDURE GetProjectByProjectId
    @Id INT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT ProjectName, ClientName, ProjectDescription, CreatedDate from
    Projects
    WHERE Id = @Id;
END
EXEC GetProjectByProjectId @Id=2

```

Output

	ProjectName	ClientName	ProjectDescription	CreatedDate
1	Go pure	Dinesh sir	Go pure Description	2023-07-21 09:34:25.543

How can we create an Update stored procedure in the SQL server?

Existing records in the database can be modified using the Update operation.

```
CREATE PROCEDURE [UpdateProject]
    @id INT
    ,@ProjectName NVARCHAR(500)
    ,@ClientName NVARCHAR(500)
    ,@ProjectManagerId NVARCHAR(500)
    ,@ProjectDescription NVARCHAR(500)
    ,@StartDate DATETIME = NULL
    ,@EndDate DATETIME = NULL
    ,@UpdatedDate DATETIME = NULL
AS
BEGIN
    UPDATE Projects
    SET ProjectName = @ProjectName
        ,ClientName = @ClientName
        ,ProjectManagerId = @ProjectManagerId
        ,ProjectDescription = @ProjectDescription
        ,StartDate = @StartDate
        ,EndDate = @EndDate
        ,UpdatedDate = getutcdate()
    WHERE Id = @Id
END

EXEC UpdateProject
    @Id=1,
    @ProjectName = 'TimeSystem Software',
    @ClientName = 'Shubham Mishra',
    @ProjectManagerId = 'A51DC085-073F-4D3A-AFC8-ACE61B89E8C8',
    @ProjectDescription = 'This is a sample TimeSystem Software.',
    @StartDate = '2023-01-20 12:45:00.000',
    @EndDate = '2023-04-20 12:45:00.000',
    @UpdatedDate= getutcdate();
```

Output

	Id	ProjectName	ClientName	ProjectManagerId	ProjectDescription	StartDate	EndDate	CreatedDate	CreatedBy	UpdatedDate
1	1	TimeSystem...	Shubham ...	A51DC085-073F-...	This is a sample T...	2023-01-2...	2023-0...	2023-07-21 ...	NULL	2023-07-21 ...
2	2	Go pure	Dinesh sir	A51DC085-073F-...	Go pure Descripti...	2023-01-2...	NULL	2023-07-21 ...	NULL	NULL
3	3	Timesheet	Bhasker sir	A51DC085-073F-...	This a sample Ti...	2023-01-2...	NULL	2023-07-21 ...	NULL	NULL

How can we create a Delete stored procedure in the SQL server?

To delete records from the database, use the Delete operation. To develop a stored process for deletion

```
CREATE PROCEDURE DeleteProjectById
    @id int
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM Projects
    WHERE id = @id;
END

EXEC DeleteProjectById @Id=2
```

Output

	Id	ProjectName	ClientName	ProjectManagerId	ProjectDescription	StartDate	EndDate	CreatedDate	UpdatedDate	IsActive	IsDelete
1	1	TimeSystem...	Shubham...	A51DC085-073F...	This is a sample ...	2023-...	2023-04...	2023-07-21 ...	2023-07-21...	1	0
2	3	Timesheet	Bhasker sir	A51DC085-073F...	This a sample Ti...	2023-...	NULL	2023-07-21 ...	NULL	1	0

To preserve accurate information and past information, gentle deletion rather than hard deletion is frequently used in database designs. Instead of physically removing records from the database, soft delete involves listing them as inactive or removed. This strategy enables previous tracking and data retrieval.

```
CREATE PROCEDURE [DeleteProject]
    @id int
AS
BEGIN
    UPDATE Projects SET IsDelete =1, IsActive =0
    WHERE Id = @id
END

EXEC DeleteProject @Id=2
```

Output

	Id	ProjectName	ClientName	ProjectManagerId	ProjectDescription	StartDate	EndDate	CreatedDate	UpdatedDate	IsActive	IsDelete
1	1	TimeSystem...	Shubham ...	A51DC085-073F...	This is a sampl...	2023-0...	2023-...	2023-07-2...	2023-07-21 ...	1	0
2	2	Go pure	Dinesh sir	A51DC085-073F...	Go pure Descri...	2023-0...	NULL	2023-07-2...	NULL	0	1
3	3	Timesheet	Bhasker sir	A51DC085-073F...	This a sample T...	2023-0...	NULL	2023-07-2...	NULL	1	0

Custom Functions

creating custom functions using T-SQL in SQL Server, both scalar-valued and table-valued functions, along with examples and how to use them in queries.

Scalar-Valued Function Example

A scalar-valued function returns a single value. Here is an example of creating a scalar-valued function that calculates a bonus based on a given salary:

Table-Valued Function Example

A table-valued function returns a table. Here is an example of creating an inline table-valued function that returns employees with a salary above a certain threshold:

This example demonstrates how to create and use both scalar-valued and table-valued functions in SQL Server. The CalculateBonus function calculates a 10% bonus based on the salary, and the HighEarningEmployees function returns all employees earning more than a specified amount.

Create the Employees table

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName NVARCHAR(50),  
    LastName NVARCHAR(50),  
    Salary DECIMAL(18, 2)  
);
```

Insert sample data

```
INSERT INTO Employees (EmployeeID, FirstName, LastName, Salary)  
VALUES  
(1, 'John', 'Doe', 45000),  
(2, 'Jane', 'Smith', 60000),  
(3, 'Jim', 'Brown', 75000);
```

Create scalar-valued function

```
CREATE FUNCTION dbo.CalculateBonus (@Salary DECIMAL)  
RETURNS DECIMAL  
AS  
BEGIN  
    RETURN @Salary * 0.10;  
END;
```

Use scalar-valued function in a query

```
SELECT  
    EmployeeID,  
    Salary,  
    dbo.CalculateBonus(Salary) AS Bonus  
FROM  
    Employees;
```

Create table-valued function

```
CREATE FUNCTION dbo.HighEarningEmployees (@MinSalary DECIMAL)  
RETURNS TABLE  
AS  
RETURN  
(  
    SELECT  
        EmployeeID,  
        FirstName,  
        LastName,
```

```
        Salary
    FROM
        Employees
    WHERE
        Salary > @MinSalary
);
```

Use table-valued function in a query

```
SELECT
    *
FROM
    dbo.HighEarningEmployees (50000);
```

9

Database backup With Table and Data

Overview

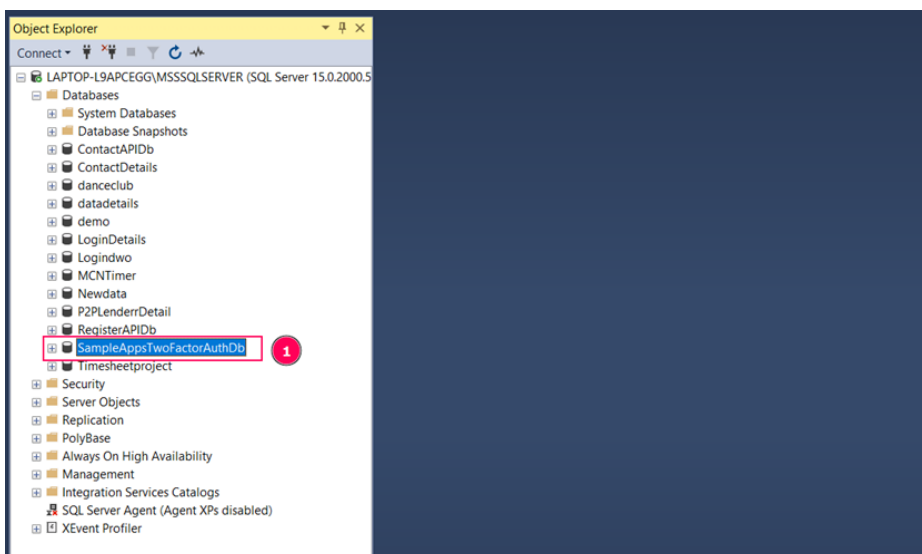
In this chapter, we explore how to create comprehensive database backups, including tables and data, in SQL Server. You'll learn the step-by-step process to generate scripts for all database objects, ensuring data integrity and enabling easy transfer between databases.

Create Database backup With Table and Data In SQL Server

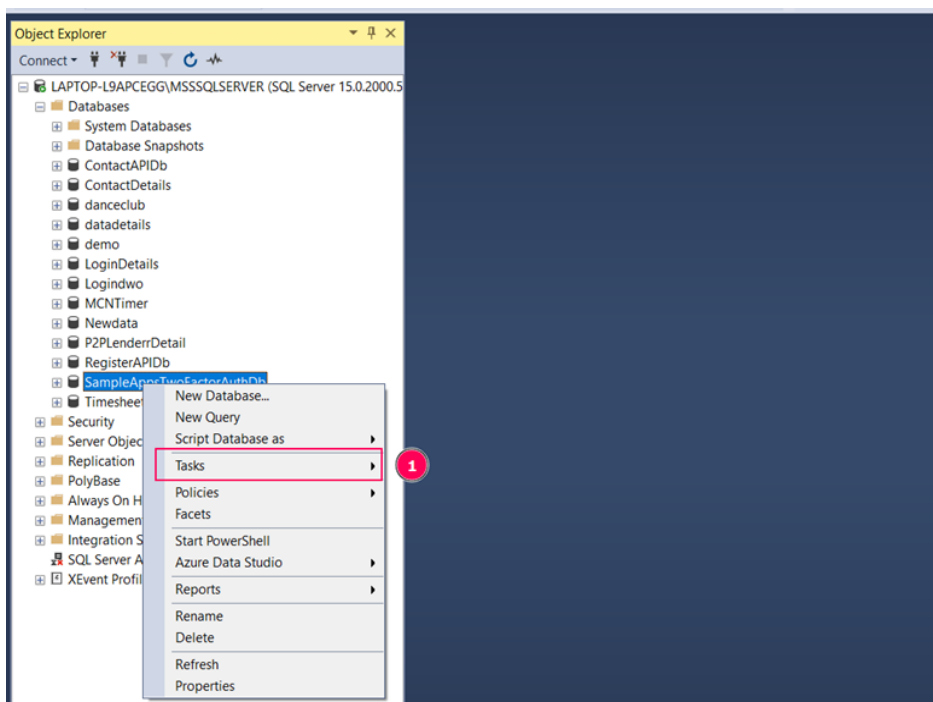
A script is created for all the tables, views, functions, stored procedures, indexes, and other objects of a chosen database when you do generate script actions. When backup failed to restore a database from a lower version database to an upper version due to different versions, it might be helpful in such a situation. In this situation, you can create a script using the database schema and data using generate scripts, then run that script against the target database. It can also be helpful if you only require one table's worth of data. You can script just one table's worth of data using the produce scripts option. Let's look at how to create a script for a whole database or just a few database objects.

The steps are as follows,

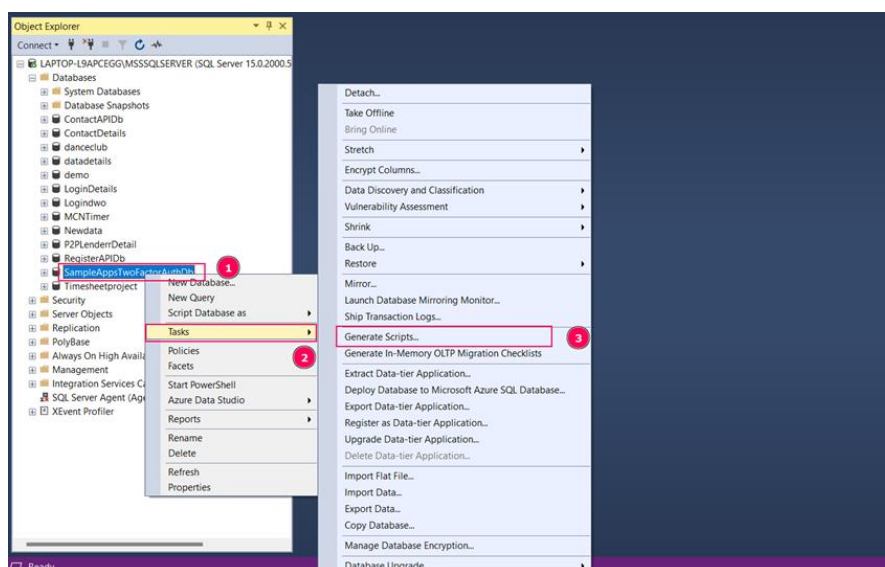
- **Step 1:** Open SQL Server open your object explorer select the database



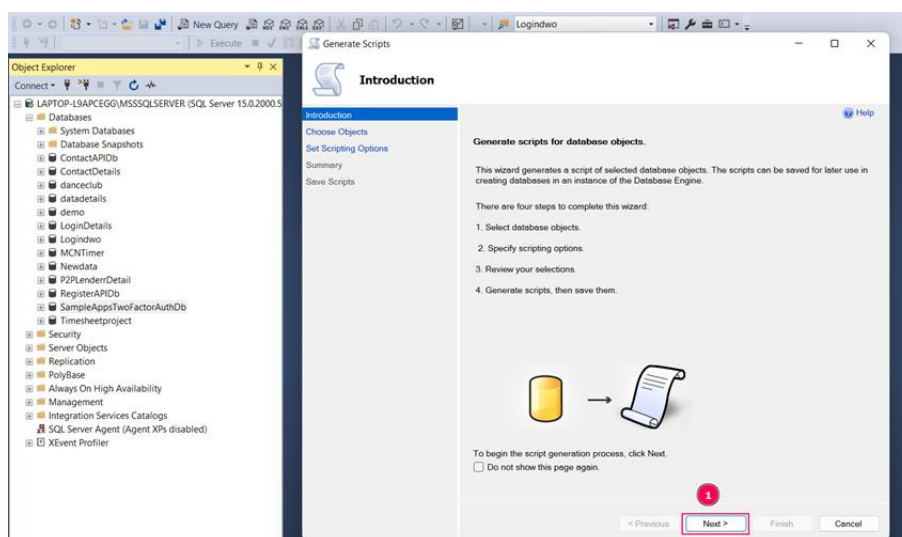
- **Step 2:** Right-click on Database select the Tasks option



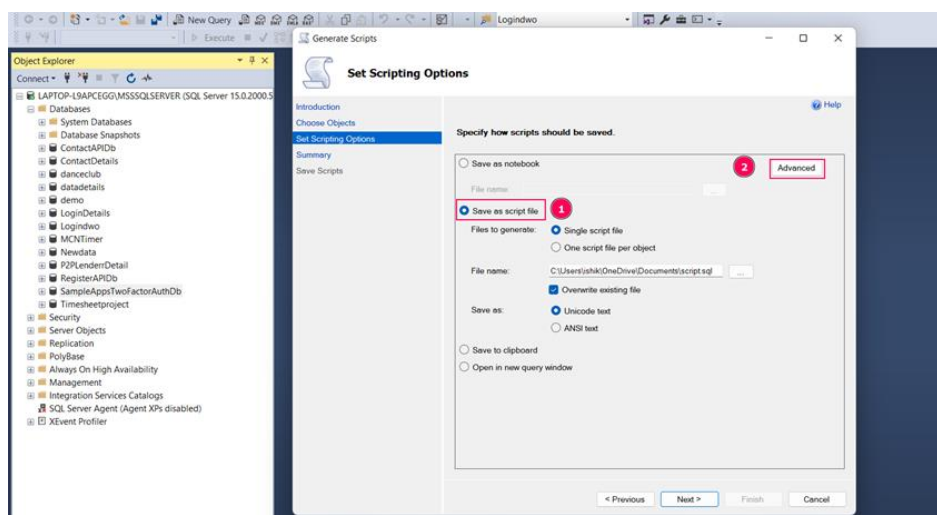
- **Step 3:** Right-click on Task and select generate Scripts



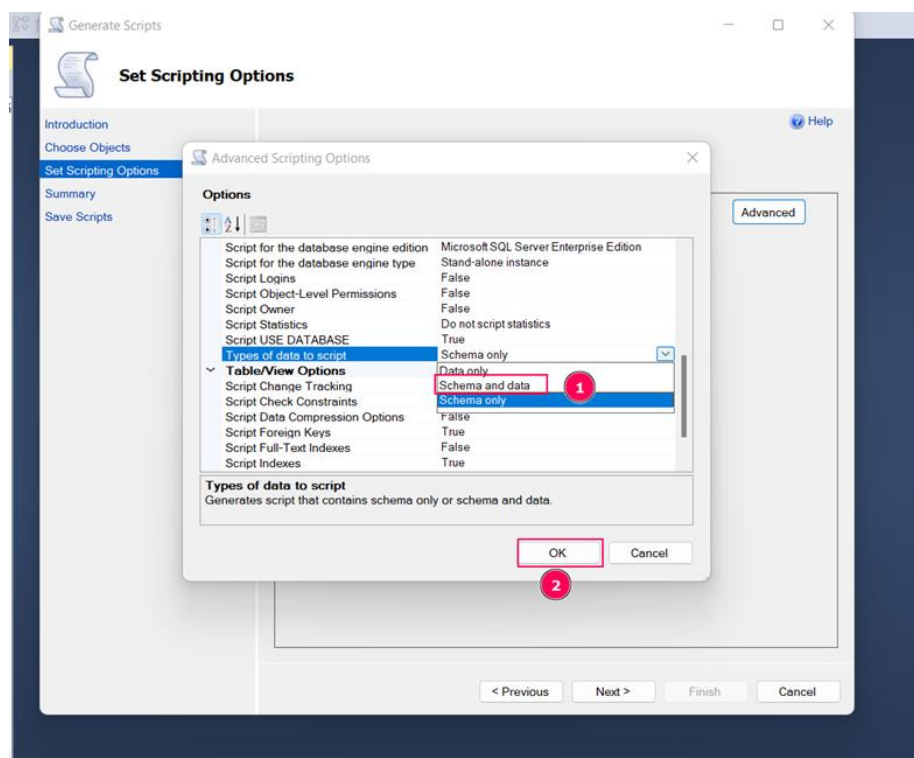
- **Step 4:** Press the next button to continue the process



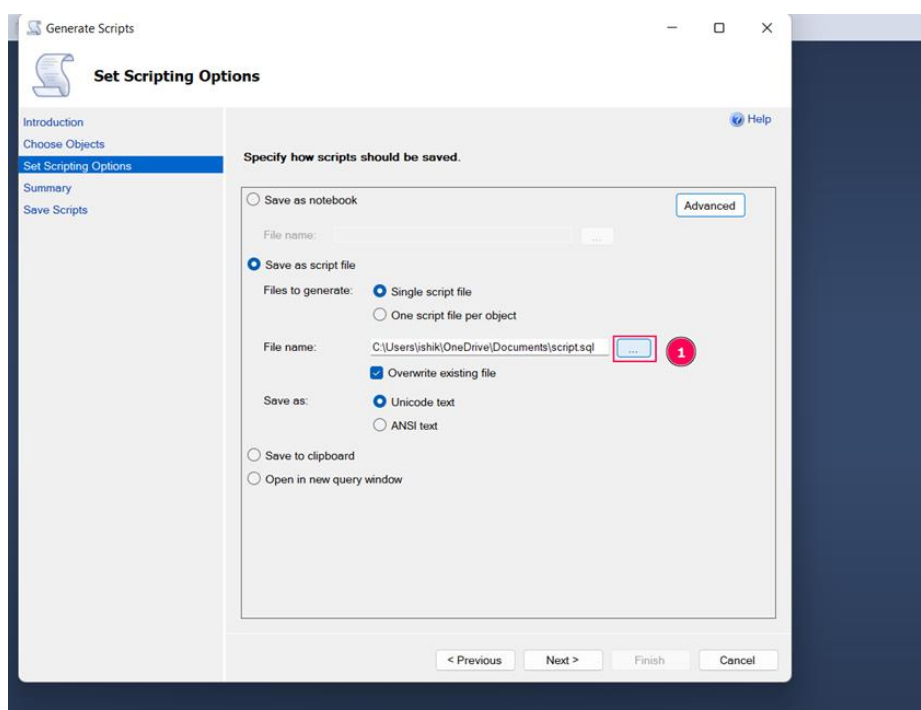
- **Step 5:** Firstly, select save as script file and then click on the advanced option



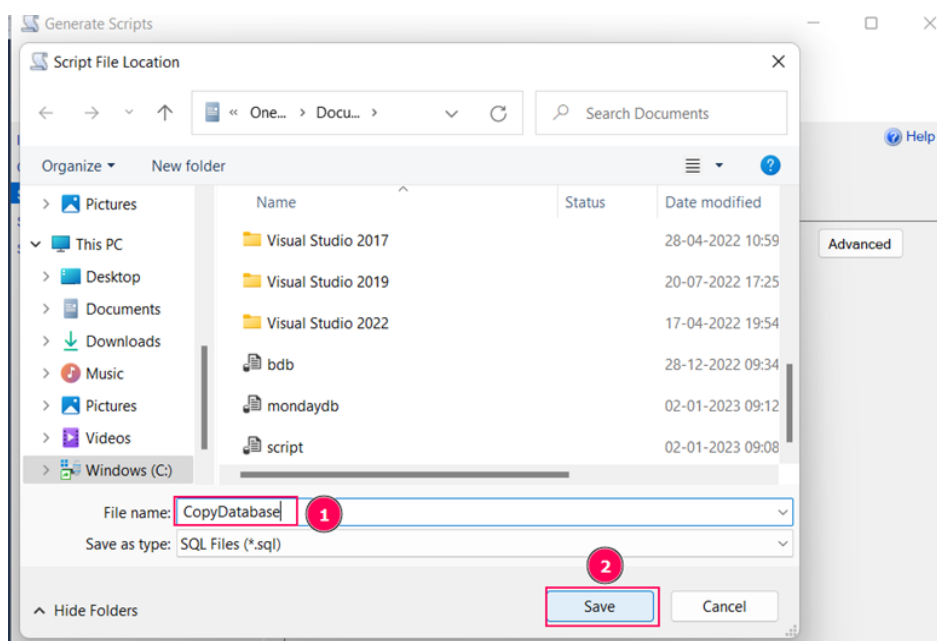
- **Step 6:** Now there is an option to select Types of Data to script which contains three options - Schema only, Data only, and Schema and Data choice depending on requirements. Currently, I am choosing Schema and Data and clicking the ok button



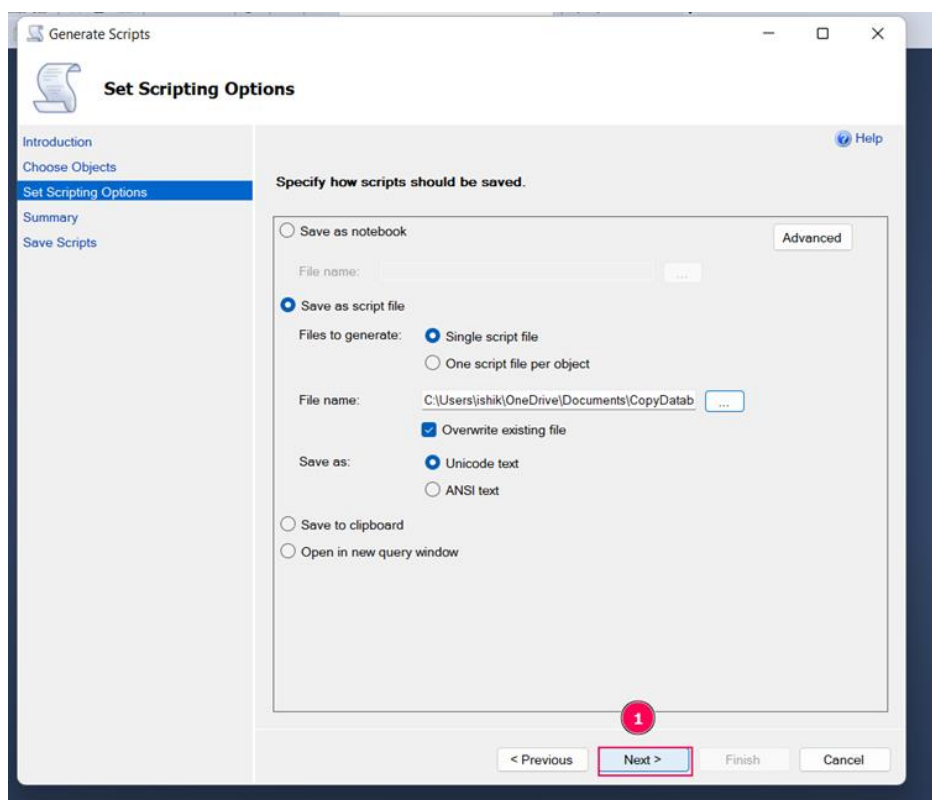
- **Step 7:** After selecting save as script file, click on the first option to save the file.



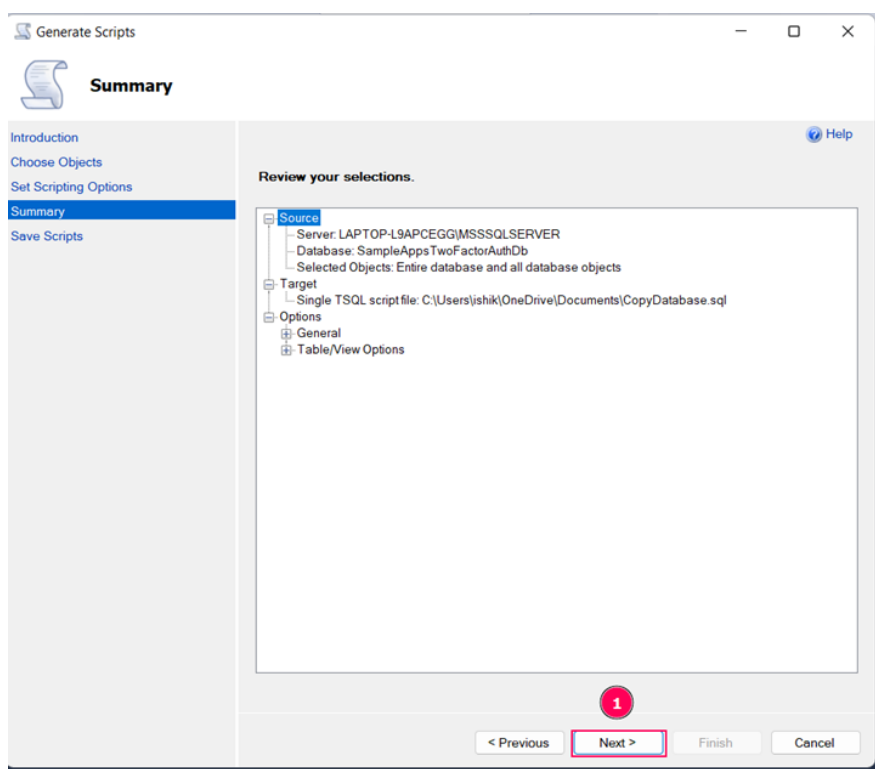
- **Step 8:** We must provide the script file name and click on the save button.



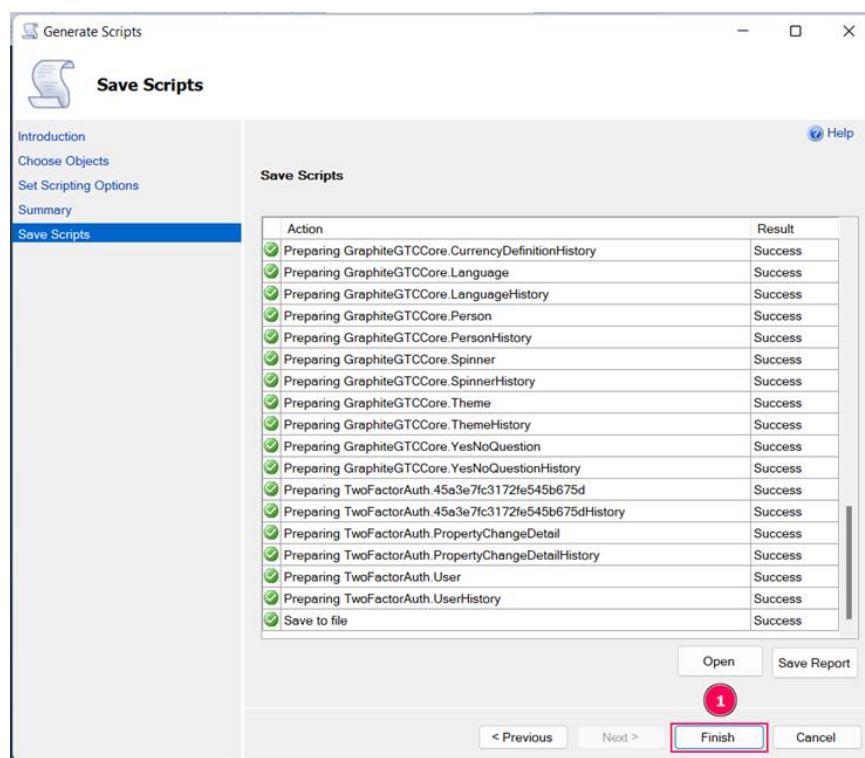
- **Step 9:** Now after all the steps, we move forward and click on the next button



- **Step 10:** Again, click on the next button



- **Step 11:** In this final step save and generate the script





OUR MISSION

Free Education is Our Basic Need! Our mission is to empower millions of developers worldwide by providing the latest unbiased news, advice, and tools for learning, sharing, and career growth. We're passionate about nurturing the next young generation and help them not only to become great programmers, but also exceptional human beings.

ABOUT US

CSharp Inc, headquartered in Philadelphia, PA, is an online global community of software developers. C# Corner served 29.4 million visitors in year 2022. We publish the latest news and articles on cutting-edge software development topics. Developers share their knowledge and connect via content, forums, and chapters. Thousands of members benefit from our monthly events, webinars, and conferences. All conferences are managed under Global Tech Conferences, a CSharp Inc sister company. We also provide tools for career growth such as career advice, resume writing, training, certifications, books and white-papers, and videos. We also connect developers with their potential employers via our Job board. Visit [C# Corner](#)

MORE BOOKS

